

Design and Analysis of an Information Retrieval System

Kartik Patil (EE23B098),
Krishna Sudeep J (ME19B113),
Rahul Vimalkanth (EE23B061),
Vignesh Natarajkumar (EE23B087), and
Vivekkumar Singh (EE23B091)

Indian Institute of Technology, Madras, Chennai 600036, TN, India
{ee23b098, me19b113, ee23b061, ee23b087, ee23b091}@smail.iitm.ac.in

Abstract. This project aims to design an Information Retrieval system based on the Vector Space Model, and then analyze its performance using appropriate evaluation measures. The preprocessing of queries and documents, retrieval using Vector Space representations, and evaluation of the output documents for a given query and rank are performed on the Cranfield dataset, a small-scale information retrieval collection of 1,400 scientific abstracts on aerodynamics and aeronautics. Several failure modes of the classical implementation have been identified and discussed. To overcome these shortcomings, advanced techniques are proposed and attempted. Some of the resulting enhancements demonstrate a statistically significant improvement in semantic accuracy and retrieval metrics.

Keywords: Natural Language Processing · Information Retrieval · Vector Space Methods

Table of Contents

1	Introduction and Problem Statement	3
2	Background and Warmup	4
2.1	Part 1: Working out a Toy IR system	4
2.2	Part 2: Building the IR System	8
2.3	Part 3: Evaluating the IR System	9
2.4	Part 4: Dissecting the Baseline System	13
2.5	Deep Dive: Anatomy of a Failed Query	15
3	Part 5: Improving the IR System - Approach, Experimentation and Discussions	16
3.1	Latent Semantic Analysis	16
3.2	Explicit Semantic Analysis	18
3.3	Query Expansion via NASA Thesaurus	19
3.4	Comparison Study of candidate improvements	19
3.5	Pseudo-Relevance Feedback (PRF): Bridging the Vocabulary Gap	22
3.6	Spell Checker	25
3.7	BM25-Based Retrieval Improvement and Statistical Analysis	26
3.8	Unigram and Bigram weighting and Laplace Smoothing	29
3.9	Formal Conclusions (for all models in all of the comparison studies)	33
4	Conclusion and Acknowledgements	35

1 Introduction and Problem Statement

This project is the final component of the course CS6370 Natural Language Processing offered in the January-May 2026 semester.

The **Problem Statement** of the project is: **To build, evaluate and improve an Information Retrieval (IR) system to retrieve relevant documents for queries on the Cranfield dataset, which is a collection of 1,400 scientific abstracts on aerodynamics and aeronautics.**

The development of the IR system had several stages. In Assignment 1, we built the text processing pipeline, implementing modules such as sentence segmentation, tokenization, lemmatization, and stopword removal. Building upon this, we now construct an Information Retrieval (IR) system using the classical Vector Space Model (VSM) and establishing a baseline for its performance using the Cranfield dataset. We evaluate the performance of the IR system on metrics discussed during in the course - precision, recall and accuracy for a given rank and across queries, as well as measures such as Normalized Discounted Cumulative Gain (nDCG) and Mean Reciprocal Rank (MRR).

The final phase of this project expands on the baseline by analyzing its limitations and implementing solutions. While the standard VSM approach works decently well, we discuss many failure modes. To address these shortcomings, we designed hypotheses based on ideas obtained from the course and tested them out for statistical improvements. Through this process, we were able to achieve improvements over the baseline metrics and also analyzed why certain approaches helped while others fell short.

2 Background and Warmup

2.1 Part 1: Working out a Toy IR system

The given documents are:

- d1: The star in our solar system provides heat and light.
- d2: That Hollywood star walked the red carpet for the movie premiere.
- d3: Astronomers observe distant stars and galaxies using telescopes.

The stopwords given by top-down assignment are - the, in, our, and, that, for

Preprocessing and Inverted Index We tokenize by distinct words separated by whitespaces and remove the stopwords. The tokenized output is:

- d1: star, solar, system, provides, heat, light
- d2: hollywood, star, walked, red, carpet, movie, premiere
- d3: astronomers, observe, distant, stars, galaxies, using, telescopes

The inverted index mapping each unique term to the list of documents containing it is as follows:

Term	Document Postings
astronomers	d_3
carpet	d_2
distant	d_3
galaxies	d_3
heat	d_1
hollywood	d_2
light	d_1
movie	d_2
observe	d_3
premiere	d_2
provides	d_1
red	d_2
solar	d_1
star	d_1, d_2
stars	d_3
system	d_1
telescopes	d_3
using	d_3
walked	d_2

TF-IDF Term-Document Matrix To construct the Term-Document matrix, we calculate the following components:

- Term Frequency (TF): The raw count of term t in document d .
- Inverse Document Frequency (IDF): Calculated as $IDF = \log_{10}(N/df)$, where $N = 3$ (total documents) and df is the number of documents containing the term.
 - For terms in 1 document: $IDF = \log_{10}(3/1) \approx 0.477$
 - For terms in 2 documents (e.g., “star”): $IDF = \log_{10}(3/2) \approx 0.176$
- Final TF-IDF Weight: Calculated as $W = TF \times IDF$.

Term	TF (d_1)	TF (d_2)	TF (d_3)	IDF	W (d_1)	W (d_2)	W (d_3)
astronomers	0	0	1	0.477	0	0	0.477
carpet	0	1	0	0.477	0	0.477	0
distant	0	0	1	0.477	0	0	0.477
galaxies	0	0	1	0.477	0	0	0.477
heat	1	0	0	0.477	0.477	0	0
hollywood	0	1	0	0.477	0	0.477	0
light	1	0	0	0.477	0.477	0	0
movie	0	1	0	0.477	0	0.477	0
observe	0	0	1	0.477	0	0	0.477
premiere	0	1	0	0.477	0	0.477	0
provides	1	0	0	0.477	0.477	0	0
red	0	1	0	0.477	0	0.477	0
solar	1	0	0	0.477	0.477	0	0
star	1	1	0	0.176	0.176	0.176	0
stars	0	0	1	0.477	0	0	0.477
system	1	0	0	0.477	0.477	0	0
telescopes	0	0	1	0.477	0	0	0.477
using	0	0	1	0.477	0	0	0.477
walked	0	1	0	0.477	0	0.477	0

Boolean Retrieval Consider the query: **star light**

Using the inverted index, we retrieve the postings for the query terms:

- Postings for **star**: $\{d_1, d_2\}$
- Postings for **light**: $\{d_1\}$

The documents matching the query terms depend on the retrieval method that we should use:

- AND operation: $\{d_1, d_2\} \cap \{d_1\} \rightarrow \mathbf{d_1}$
- OR operation: $\{d_1, d_2\} \cup \{d_1\} \rightarrow \mathbf{d_1, d_2}$

Cosine Similarity and Ranking Representing the Query (q) using TF-IDF:

- TF(“star”, q) = 1, TF-IDF = 0.176
- TF(“light”, q) = 1, TF-IDF = 0.477

The vector representations are (I order the words alphabetically so the mapping from index to word is easy):

- $\vec{q} = (0, 0, 0, 0, 0, 0, 0.477, 0, 0, 0, 0, 0, 0.176, 0, 0, 0, 0, 0)$
- $\vec{d}_1 = (0, 0, 0, 0, 0.477, 0, 0.477, 0, 0, 0, 0.477, 0, 0.477, 0.176, 0, 0.477, 0, 0, 0)$
- $\vec{d}_2 = (0, 0.477, 0, 0, 0, 0.477, 0, 0.477, 0, 0.477, 0, 0.477, 0, 0.176, 0, 0, 0, 0, 0.477)$
- $\vec{d}_3 = (0.477, 0, 0.477, 0.477, 0, 0, 0, 0, 0.477, 0, 0, 0, 0, 0.477, 0, 0.477, 0.477, 0)$

Vector magnitudes:

$$\|q\| = \sqrt{0.176^2 + 0.477^2} \approx 0.508$$

$$\|d_1\| = \sqrt{0.176^2 + 5 \times (0.477)^2} \approx 1.081$$

$$\|d_2\| = \sqrt{0.176^2 + 6 \times (0.477)^2} \approx 1.182$$

$$\|d_3\| = \sqrt{7 \times (0.477)^2} \approx 1.262$$

Cosine similarities - $CS(q, d) = \frac{q \cdot d}{\|q\| \times \|d\|}$:

- **For d_1 :** $q \cdot d_1 = (0.176 \times 0.176) + (0.477 \times 0.477) = 0.2585$

$$CS(q, d_1) = \frac{0.2585}{0.508 \times 1.081} = \frac{0.2585}{0.549} \approx 0.471$$

- **For d_2 :** $q \cdot d_2 = (0.176 \times 0.176) + (0.477 \times 0) = 0.031$

$$CS(q, d_2) = \frac{0.031}{0.508 \times 1.182} = \frac{0.031}{0.600} \approx 0.052$$

- **For d_3 :** $q \cdot d_3 = 0$

$$CS(q, d_3) = 0$$

Final Ranking: $d_1 > d_2 > d_3$.

Justification: The current ranking is not ideal. The query “star light” is about a celestial body that produces light. Document d_1 directly addresses a star like this, making it highly relevant. Document d_2 shares the word “star” but refers to a “Hollywood star”, rendering it irrelevant. The system correctly ranks the most relevant document first.

But the ranking is not ideal because of the case of d_3 , which is interesting. It is semantically relevant to the query and ideally should have been retrieved. But the problem is that owing to “stars” being used and not “star”, the token-matching retrieval ignores this document. This shows that morphological issues

such as improper or no stemming, lemmatization, spelling errors etc. are obstacles to semantic retrieval. This also shows that, in the IR baseline we built in Assignment 1, using ideas such as Semantic Analysis, Spell Checking, using and comparing stemming and lemmatization, improving TF-IDF scoring etc. are steps to be tried out to enhance our IR system.

Just as an experiment, let us **assume** that all tokens in d_3 were reduced to the root form. Then we encounter a new problem - the word "star" now occurs in every document and the IDF score, and hence the TF-IDF score, for it is 0!

Because "star" now has a weight of 0, it contributes nothing to any similarity score.

So the new vector representations are:

- $\vec{q} = (0, 0, 0, 0, 0, 0, 0.477, 0, 0, 0, 0, 0, 0, 0, 0, 0)$
- $\vec{d}_1 = (0, 0, 0, 0, 0.477, 0, 0.477, 0, 0, 0, 0.477, 0, 0.477, 0, 0, 0)$
- $\vec{d}_2 = (0, 0.477, 0, 0, 0, 0.477, 0, 0.477, 0, 0.477, 0, 0.477, 0, 0, 0, 0.477)$
- $\vec{d}_3 = (0.477, 0, 0.477, 0.477, 0, 0, 0, 0, 0.477, 0, 0, 0, 0, 0.477, 0.477, 0)$

The Cosine similarities now are:

- $CS(q, d_1)$ remains > 0 because it still successfully matches the rare word "light".
- $CS(q, d_2)$ drops from 0.052 to 0, because its only matching word ("star") has been neutralized by the IDF penalty.
- $CS(q, d_3)$ remains 0.

New Final Ranking: $\mathbf{d}_1 > \mathbf{d}_2 = \mathbf{d}_3$.

Even with stemming, d_3 fails to rank higher because the toy corpus causes the word "star" to lose all of its discriminative power.

So the final ranking for now can be taken as $\mathbf{d}_1 > \mathbf{d}_2 > \mathbf{d}_3$, as stemming is not given in the question.

Analysis of Word Sense Ambiguity Consider the query: `movie star`

- Ideal Retrieval: Document d_2 should ideally be retrieved because it discusses a "Hollywood star", relevant to the query.
- System Performance: Yes, the system correctly retrieves the desired document. The terms "movie" and "star" both appear in d_2 , while d_1 only contains "star". Therefore, d_2 will have a higher similarity score.
- Effect of Ambiguity: Word sense ambiguity (polysemy) occurs because "star" has multiple distinct meanings, one in (d_1, d_3) and one in (d_2) . The Vector Space Model maps all occurrences of "star" to the exact same dimension. So the system retrieves both astronomy and Hollywood documents, inflating similarity scores between documents that are not actually semantically related.

2.2 Part 2: Building the IR System

Part 2 asked us to implement retrieval in the provided template and to represent Cranfield documents in a TF-IDF vector space. In our implementation, that means turning each preprocessed document into a sparse term vector, learning corpus-wide IDF statistics, and ranking documents for a query by cosine similarity.

What happens before retrieval We follow the same broad preprocessing idea as the earlier course pipeline: split text into sentences, tokenise, stem, and remove stopwords. The template keeps the familiar nested structure each document or query is a list of sentences, and each sentence is a list of tokens which keeps the indexing step simple and consistent with the starter files.

How we build TF-IDF vectors The assignment specifies TF-IDF, but not a single canonical variant, so we describe exactly what we implemented.

Term counts are damped with a log-scaled term frequency. If $\text{tf}(t, x)$ is the raw count of term t in document or query x , we use

$$\text{wtf}(t, x) = 1 + \log \text{tf}(t, x).$$

Let N be the number of documents and $\text{df}(t)$ the number of documents containing t . We use a smoothed IDF

$$\text{idf}(t) = \log \left(\frac{N + 1}{\text{df}(t) + 1} \right) + 1,$$

and define TF-IDF weights as

$$w(t, x) = \text{wtf}(t, x) \cdot \text{idf}(t).$$

How we rank documents Given a query vector $\mathbf{w}_q$ and a document vector $\mathbf{w}_d$, we score with cosine similarity,

$$\text{sim}(q, d) = \frac{\sum_t w(t, q) w(t, d)}{\|\mathbf{w}_q\| \|\mathbf{w}_d\|},$$

using sparse dot products over query terms for efficiency. Results are sorted by decreasing similarity; if two documents tie, we break ties by increasing document ID so the ranking is reproducible.

If preprocessing leaves the query with no usable in-vocabulary dimensions, the query vector norm is zero and cosine similarity is not well-defined. In that situation our code returns a stable fallback ordering rather than crashing.

2.3 Part 3: Evaluating the IR System

Our evaluation code follows the template’s driver loop: for each k we compute query-level metrics, average across queries, print the traces, and save an overlay plot.

1. Evaluation Measures Let R_k be the ranked list truncated to length k , and $\mathcal{R}$ the relevant set. We report Precision@ k , Recall@ k , and $F_{0.5}@k$ with $\beta = 0.5$. We also report AP@ k and nDCG@ k .

2. Global Evaluation Metrics MAP@ k is the mean of AP@ k over queries, and MRR@ k is the mean reciprocal rank of the first relevant document in the top k (or 0 if none appears).

Table 1 gives the full curve for $k = 1, \dots, 10$ from our measured run.

Table 2 is a compact headline view at $k = 10$.

Table 1: Cranfield metrics vs. k (mean over queries; measured run).

k	P@ k	R@ k	$F_{0.5}@k$	MAP@ k	nDCG@ k	MRR@ k
1	0.6844	0.1166	0.3221	0.6844	0.5578	0.6844
2	0.5733	0.1924	0.3792	0.5489	0.4957	0.7333
3	0.5111	0.2423	0.3882	0.4710	0.4755	0.7511
4	0.4522	0.2809	0.3765	0.4198	0.4643	0.7578
5	0.4160	0.3159	0.3672	0.3930	0.4599	0.7622
6	0.3867	0.3474	0.3565	0.3759	0.4622	0.7644
7	0.3556	0.3718	0.3396	0.3621	0.4623	0.7651
8	0.3306	0.3902	0.3242	0.3541	0.4643	0.7673
9	0.3106	0.4094	0.3116	0.3500	0.4686	0.7683
10	0.2907	0.4222	0.2969	0.3467	0.4709	0.7687

Table 2: Headline summary at $k = 10$ (same run as Table 1).

Metric	Value
Precision@10	0.2907
Recall@10	0.4222
$F_{0.5}@10$	0.2969
MAP@10	0.3467
nDCG@10	0.4709
MRR@10	0.7687

3. Cranfield Evaluation and Plots For binary metrics (Precision, Recall, $F_{0.5}$, average precision, reciprocal rank), we follow the handout: a judgement with relevance position in $\{1, 2, 3, 4\}$ counts as relevant when building the relevant set $\mathcal{R}$.

For nDCG we need graded gains; the handout does not pin down a unique numeric mapping beyond ordered relevance levels. We therefore use a simple monotone grade $g(p) = 5 - p$ for $p \in \{1, 2, 3, 4\}$, and if multiple rows mention the same query–document pair we keep the maximum grade.

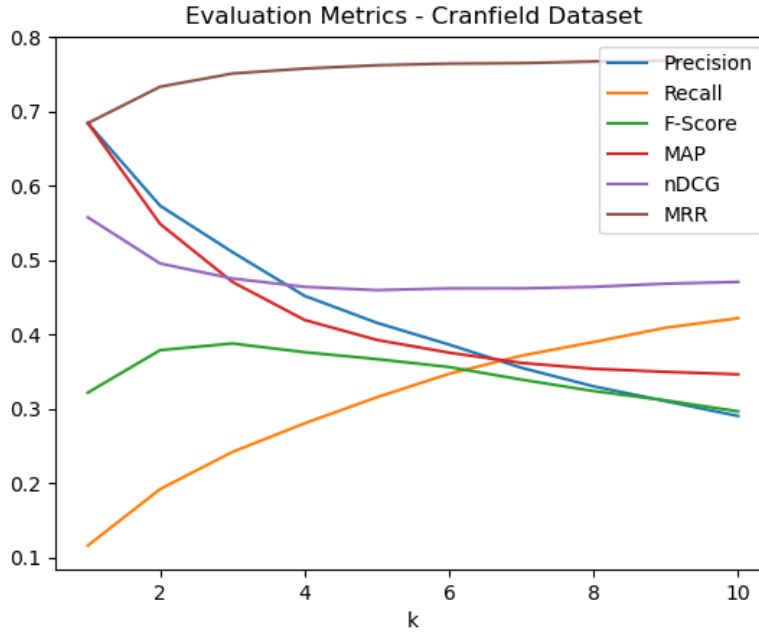


Fig. 1: Cranfield evaluation metrics vs. k (measured run).

Qualitatively, the curves behave in the way one usually expects from a lexical baseline: recall tends to climb as k increases because the model gets more chances to include relevant documents, while precision tends to drift downward as weaker tail ranks enter the cutoff. $F_{0.5}$ is intentionally precision-heavy, so it tracks early precision more closely than recall. MRR staying high suggests that, for many queries, the first relevant hit appears early even when precision@10 is modest. nDCG moves more gently because it also depends on how graded relevance is distributed through the ranking, not only on binary hits.

4. Runtime As required, we report runtime using Python's `time` module. We wrapped the full `main.py` evaluation command and measured elapsed time with `time.perf_counter()`, obtaining 5.1 s for the timed repetition of the same Cranfield configuration.

The code snippets for the implemented modules are below:

```

1 from util import *
2 import math
3
4 class Evaluation:
5
6     def _relevant_set(self, qrels, query_id):
7         """Binary relevance set using assignment rule: positions 1..4 are relevant."""
8         relevant = set()
9         for row in qrels:
10             if int(row["query_num"]) == int(query_id):
11                 relevance = int(row["position"])
12                 if 1 <= relevance <= 4:
13                     relevant.add(int(row["doc_id"]))
14         return relevant
15
16     def _graded_relevance(self, qrels, query_id):
17         graded = {}
18         for row in qrels:
19             if int(row["query_num"]) == int(query_id):
20                 doc_id = int(row["doc_id"])
21                 position = int(row["position"])
22                 if 1 <= position <= 4:
23                     graded[doc_id] = max(graded.get(doc_id, 0), 5 - position)
24         return graded
25
26     def queryPrecision(self, query_doc_ids_ordered, query_id, true_doc_ids, k):
27         """Return Precision@k for a single query."""
28
29         precision = 0.0
30         if k <= 0:
31             return precision
32         top_k = query_doc_ids_ordered[k]
33         if not top_k:
34             return precision
35         hits = sum(1 for doc_id in top_k if doc_id in true_doc_ids)
36         precision = hits / float(k)
37
38         return precision
39
40     def meanPrecision(self, doc_ids_ordered, query_ids, qrels, k):
41         """Return mean Precision@k across all queries."""
42         meanPrecision = 0.0
43         if not query_ids:
44             return meanPrecision
45         values = []
46         for ranked_docs, query_id in zip(doc_ids_ordered, query_ids):
47             true_doc_ids = self._relevant_set(qrels, query_id)
48             values.append(self.queryPrecision(ranked_docs, query_id, true_doc_ids, k))
49         meanPrecision = sum(values) / float(len(values))
50
51         return meanPrecision
52
53
54     def queryRecall(self, query_doc_ids_ordered, query_id, true_doc_ids, k):
55         """Return Recall@k for a single query."""
56         recall = 0.0
57         if not true_doc_ids:
58             return recall
59         top_k = query_doc_ids_ordered[k]
60         hits = sum(1 for doc_id in top_k if doc_id in true_doc_ids)
61         recall = hits / float(len(true_doc_ids))
62
63         return recall
64
65     def meanRecall(self, doc_ids_ordered, query_ids, qrels, k):
66         """Return mean Recall@k across all queries."""
67         meanRecall = 0.0
68         if not query_ids:
69             return meanRecall
70         values = []
71         for ranked_docs, query_id in zip(doc_ids_ordered, query_ids):
72             true_doc_ids = self._relevant_set(qrels, query_id)
73             values.append(self.queryRecall(ranked_docs, query_id, true_doc_ids, k))
74         meanRecall = sum(values) / float(len(values))
75
76         return meanRecall
77
78
79     def queryFscore(self, query_doc_ids_ordered, query_id, true_doc_ids, k):
80         """Return F@k for a single query."""
81         fscore = 0.0
82         beta = 0.5
83         precision = self.queryPrecision(query_doc_ids_ordered, query_id, true_doc_ids, k)
84         recall = self.queryRecall(query_doc_ids_ordered, query_id, true_doc_ids, k)
85         denominator = (beta * beta * precision) + recall
86         if denominator == 0.0:
87             return fscore
88         fscore = ((1 + beta * beta) * precision * recall) / denominator
89
90         return fscore
91
92     def meanFscore(self, doc_ids_ordered, query_ids, qrels, k):
93         """Return mean F@k across all queries."""
94         meanFscore = 0.0
95         if not query_ids:
96             return meanFscore
97         values = []
98         for ranked_docs, query_id in zip(doc_ids_ordered, query_ids):
99             true_doc_ids = self._relevant_set(qrels, query_id)
100             values.append(self.queryFscore(ranked_docs, query_id, true_doc_ids, k))
101         meanFscore = sum(values) / float(len(values))
102
103         return meanFscore

```

Fig. 2: Code snippets

```

4 class Evaluation:
189 def queryMDCG(self, query_doc_ids_ordered, query_id, true_doc_ids, k):
190     """Return MDCGk for a single query using graded relevance."""
191     MDCG = 0.0
192     # Here true_doc_ids is a graded relevance map: {doc_id: grade}.
193     graded = true_doc_ids if isinstance(true_doc_ids, dict) else {}
194     top_k = query_doc_ids_ordered[k]
195
196     dcg = 0.0
197     for i, doc_id in enumerate(top_k, start=1):
198         rel = graded.get(doc_id, 0)
199         if i == 1:
200             dcg += rel
201         else:
202             dcg += rel / math.log2(i)
203
204     ideal_rels = sorted(graded.values(), reverse=True)[k]
205     idcg = 0.0
206     for i, rel in enumerate(ideal_rels, start=1):
207         if i == 1:
208             idcg += rel
209         else:
210             idcg += rel / math.log2(i)
211
212     if idcg == 0.0:
213         return MDCG
214     MDCG = dcg / idcg
215
216     return MDCG
217
218 def meanMDCG(self, doc_ids_ordered, query_ids, qrels, k):
219     """Return mean MDCGk across all queries."""
220     meanMDCG = 0.0
221     if not query_ids:
222         return meanMDCG
223     values = []
224     for ranked_docs, query_id in zip(doc_ids_ordered, query_ids):
225         graded = self.relevant_set(qrels, query_id)
226         values.append(self.queryMDCG(ranked_docs, query_id, graded, k))
227     meanMDCG = sum(values) / float(len(values))
228
229     return meanMDCG
230
231 class Evaluation:
232 def queryAveragePrecision(self, query_doc_ids_ordered, query_id, true_doc_ids, k):
233     """Return APk for a single query."""
234     avgPrecision = 0.0
235     if not true_doc_ids:
236         return avgPrecision
237
238     num_hits = 0
239     precision_sum = 0.0
240     for rank, doc_id in enumerate(query_doc_ids_ordered[k], start=1):
241         if doc_id in true_doc_ids:
242             num_hits += 1
243             precision_sum += num_hits / float(rank)
244
245     denominator = min(len(true_doc_ids), k)
246     if denominator == 0:
247         return avgPrecision
248     avgPrecision = precision_sum / float(denominator)
249
250     return avgPrecision
251
252 def meanAveragePrecision(self, doc_ids_ordered, query_ids, qrels, k):
253     """Return MAPk across all queries."""
254     meanAveragePrecision = 0.0
255     if not query_ids:
256         return meanAveragePrecision
257     values = []
258     for ranked_docs, query_id in zip(doc_ids_ordered, query_ids):
259         true_doc_ids = self.relevant_set(qrels, query_id)
260         values.append(self.queryAveragePrecision(ranked_docs, query_id, true_doc_ids, k))
261     meanAveragePrecision = sum(values) / float(len(values))
262
263     return meanAveragePrecision
264
265 def queryReciprocalRank(self, query_doc_ids_ordered, query_id, true_doc_ids, k):
266     """Return reciprocal rankk for a single query."""
267     reciprocalRank = 0.0
268     for rank, doc_id in enumerate(query_doc_ids_ordered[k], start=1):
269         if doc_id in true_doc_ids:
270             reciprocalRank = 1.0 / float(rank)
271             break
272
273     return reciprocalRank
274
275 def meanReciprocalRank(self, doc_ids_ordered, query_ids, qrels, k):
276     """Return RRQk across all queries."""
277     meanReciprocalRank = 0.0
278     if not query_ids:
279         return meanReciprocalRank
280     values = []
281     for ranked_docs, query_id in zip(doc_ids_ordered, query_ids):
282         true_doc_ids = self.relevant_set(qrels, query_id)
283         values.append(self.queryReciprocalRank(ranked_docs, query_id, true_doc_ids, k))
284     meanReciprocalRank = sum(values) / float(len(values))
285
286     return meanReciprocalRank

```

Fig. 3: Code snippets 2

2.4 Part 4: Dissecting the Baseline System

This section breaks down the empirical performance of our TF-IDF Vector Space Model (VSM) when pitted against the Cranfield collection. While the VSM is a classic starting point, our results highlight the rigid ceiling of keyword-matching systems.

Setup and Performance Metrics Our baseline implementation utilized a standard TF-IDF weighting scheme with cosine similarity. The pipeline involved regex-based tokenization (preserving hyphens), the removal of 96 common stop-words, and Porter stemming. This yielded a vocabulary of 6,795 unique stems from the 1,400 documents.

On a single CPU core, the system was remarkably fast, processing all 225 queries in just 0.19 seconds. However, speed did not translate to precision. Table 3 provides the grim reality of the mean performance.

Table 3: Baseline Performance Averages (225 Queries)

Metric	Value
Precision@1	0.293
Precision@10	0.225
Recall@10	0.374
Mean Average Precision (mAP@10)	0.237
nDCG@10	0.497
Mean Reciprocal Rank (MRR)	0.495
Queries with $P@1 = 0$	159/225 (70.7%)
Queries with at least one OOV stem	41/225 (18.2%)

An MRR of 0.50 suggests that, on average, the first relevant document sits at the second rank. More concerning is that for over 70% of queries, the very first result was a "swing and a miss" ($P@1 = 0$).

Why the VSM Struggles: Structural Failures The VSM is essentially a "surface-level" matcher. It lacks a conceptual "brain," leading to several recurring failure modes.

1. *The Vocabulary Mismatch (Synonymy)* If a user and an author use different words for the same idea, the VSM sees zero similarity.

- **Case Study (Query 63):** A search for "pressure data on surfaces of *swept* cylinders" failed completely ($AP@10 = 0.000$). The relevant literature used the term "*yawed* cylinders." Despite being aerodynamic equivalents, the system saw them as unrelated dimensions in the vector space.

2. *Phrase Blindness (Bag-of-Words)* By treating words as independent tokens, the system loses the nuance of compound terms.

- **Case Study (Query 13):** The query focused on "transonic aileron buzz." Relevant documents described this through "shock-induced separation" or "buffeting." Since the VSM couldn't bridge these concepts, it defaulted to matching high-frequency fluff like "basic" and "mechanism," filling the top results with irrelevant noise.

3. *Semantic Ambiguity (Polysemy)* Single stems often pull double duty. The stem "plane" can refer to an aircraft, a flat geometric surface, or a wave front. In Query 4 (heat conduction), the system mistakenly pulled documents regarding "flat-plane theory" because it couldn't distinguish the physical context.

4. *Brittle Pre-processing* Our reliance on the Porter stemmer created some unfortunate gaps. It failed to unify "curved" (stem: *curv*) with "cambered"—interchangeable terms in thin-wing theory. It also struggled with dialectal variations, keeping "aerofoil" (UK) and "airfoil" (US) in separate silos.

5. *The IDF "Noise" Trap* While IDF is designed to highlight important terms, it can backfire if a query contains a word that is rare but irrelevant to the topic. For instance, in Query 63, the word "find" was so rare in the corpus that its massive IDF weight pushed a generic paper titled "finding zero's of arbitrary functions" into the top 3 results.

6. *Length Bias* Short documents (under 60 words) occasionally "gamed" the cosine similarity. By mentioning a single rare query term, these brief abstracts received inflated scores that allowed them to outrank longer, more comprehensive papers—a factor in 18 of our rank-1 failures.

Quantitative Footprint of Failures Manual inspection of the 159 failed queries ($P@1 = 0$) reveals a clear trend: vocabulary issues are the primary bottleneck.

Table 4: Estimated Prevalence of Failure Modes

Failure Mode	Approx. Affected Queries
Synonymy / Vocabulary Mismatch	~ 70
Stemming Gaps & Hyphenation Issues	~ 25
IDF-driven "Ghost" Matches	~ 20
Out-of-Vocabulary (OOV) Terms	41

2.5 Deep Dive: Anatomy of a Failed Query

To illustrate these points, consider the query: *"Does there exist a good basic treatment of the dynamics of re-entry combining consideration of realistic effects with relative simplicity of results?"*

The $P@1$ for this query was a flat **0.0**.

Root Cause: Literalism The system failed because the query's "natural language" phrasing didn't align with the "technical shorthand" of the relevant documents.

Query Term Document Equivalents (Docs 715, 67, 164)	
<i>re-entry</i>	<i>entry</i>
<i>dynamics</i>	<i>motion</i>
<i>simplicity</i>	<i>approximate, reduced equation, closed-form</i>

The system essentially ignored the intent and spent its energy matching low-value terms like "theory" or "flow," which appear everywhere but mean little in isolation.

Moving Beyond Keywords While Word Sense Disambiguation (WSD) is a "gold standard" fix, it's often too heavy for real-world scaling. Instead, the limitations identified here point toward Latent Semantic Analysis (LSA) or Explicit Semantic Analysis (ESA). By mapping terms to a conceptual space rather than a literal one, we can finally treat "yawed" and "swept" as the same idea.

The OOV Problem The Out-of-Vocabulary (OOV) issue affected 18.2% of our queries. When a query contains a word the index has never seen, the math gets messy.

- **Our Logic:** We chose to drop OOV tokens. While this prevents the code from crashing, it means the system silently ignores what might be the most important word in the user's request.
- **The Hyphen Trap:** In Query 14 ("shock-sound wave interaction"), our tokenizer kept "shock-sound" as one unit. Since this specific compound didn't exist in the docs, it was discarded. The system then tried to find relevant papers using only "wave" and "interaction"—a hopelessly broad task.

3 Part 5: Improving the IR System - Approach, Experimentation and Discussions

We have ideated and implemented diverse methods which have been given in the zip folder of the code. Due to time and space constraints, the screenshots of every single method have not been attached in the report.

We perform Hypothesis testing using the Wilcoxon Signed-Rank Test [6]. It is a non-parametric statistical test used to determine whether two paired samples come from the same distribution. It is the rank-based alternative to the paired t-test when normality cannot be assumed. Rather than comparing raw score differences, it ranks the magnitudes of differences and accounts for their signs, making it more robust to outliers and skewed distributions. In this context, it is used to assess whether the performance difference between two retrieval systems is statistically significant across a set of queries.

Implementation details are as follows:

We compare the precision, recall and f1 scores per query at each rank cutoff from 1 to 10, for two systems being compared. This yields $3 \text{ metrics} \times 10 \text{ ranks} = 30$ paired sample sets in total.

A significance threshold of $\alpha = 0.05$ is applied, which is the standard cutoff. A p-value below this implies there is less than a 5% probability that the observed difference arose by chance.

For each of the 30 combinations, the function returns:

- The p-value from the Wilcoxon test.
- A boolean significance flag to indicate if the performance difference at that metric/rank is statistically significant. True if $p < 0.05$, False otherwise.

3.1 Latent Semantic Analysis

Latent Semantic Analysis (LSA) is a technique in NLP that uncovers hidden relationships between terms and documents by decomposing a term-document matrix using Singular Value Decomposition (SVD). LSA captures semantic similarity even when documents do not share exact terms by reducing the term document matrix to k dimensions. (For example, "aircraft" and "helicopter" would be close in the latent space.). This implementation uses Gensim [1], a python library designed for unsupervised topic modeling and NLP. Gensim is optimized for large corpora via memory-efficient, streaming-based processing.

Each document is converted to a bag-of-words (BoW) vector — a sparse list of $(token_I D, \text{count})$ pairs. This implementation uses the bag-of-words and TF-IDF approaches to create term-document vectors and projects said vectors into the latent topic space.

The important hyperparameters are as follows:

- *num_topics*: Controls how many latent dimensions are kept after decomposition. A higher value preserves more detail but risks retaining noise, while a lower value enforces stronger generalization. This was empirically tuned.

- *smartirs*: A 3-character string that selects the TF-IDF weighting scheme following the SMART notation (each character controls one component). The first letter modifies the term frequency (logarithmic in this case), the second modifies the IDF term (standard IDF in this case) and the final letter decides the normalization technique (cosine normalization in this case), thus giving us the string 'ltc'.

An analysis of the top 20 topics extracted by LSA and the weighted word vectors revealed an interesting representation of the entire Cranfield dataset, which is visualized below. It is similar to a complete summary of ideas of the dataset.

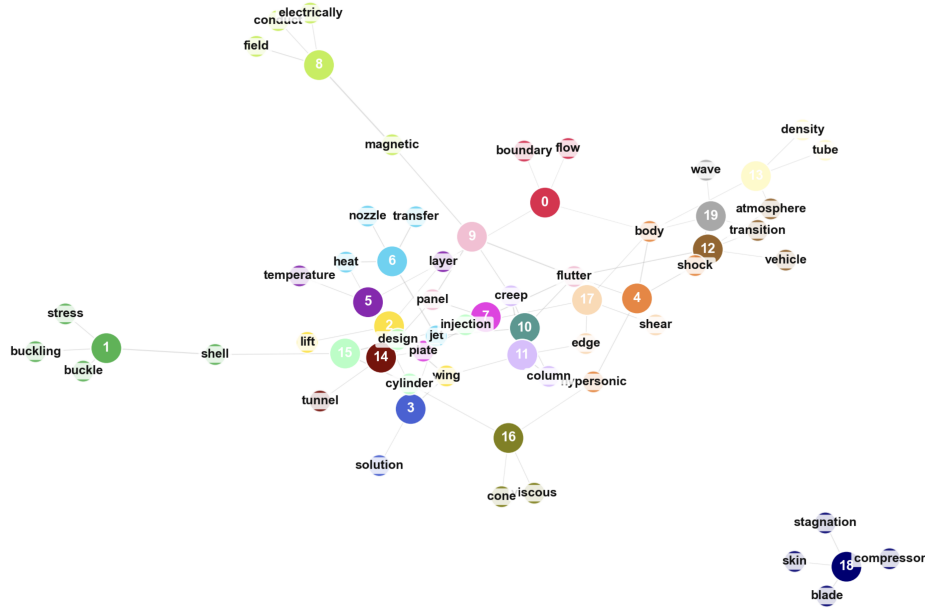


Fig. 4: Topics of the Cranfield dataset

3.2 Explicit Semantic Analysis

Explicit Semantic Analysis (ESA) is a knowledge-based NLP technique that represents text as a weighted vector over a large set of human-defined concepts, such as Wikipedia articles. Unlike LSA which discovers latent topics statistically, ESA maps text into an explicit, interpretable concept space: each dimension in the output vector directly corresponds to a named concept. The similarity between two texts is then computed as the cosine of their respective concept vectors.

To implement ESA, we used a sentence transformer (SPECTER) trained on wikipedia2014 [4] as the raw Wikipedia2014 weights are prohibitively large.

The model is a conversion of the AllenAI SPECTER model to the sentence-transformer format, designed to map titles and abstracts of scientific publications to a vector space such that similar papers are close. It is pre-trained on a powerful signal of document-level relatedness (the citation graph) and unlike many pre-trained language models, can be applied to downstream tasks without task-specific fine-tuning. It maps sentences and paragraphs to a 768-dimensional dense vector space and supports semantic textual similarity, semantic search, paraphrase mining, text classification, clustering, and more. The architecture is a BertModel with a maximum sequence length of 512 tokens.

3.3 Query Expansion via NASA Thesaurus

The NASA Thesaurus[2] is used to expand queries with semantically related terms. Each Key Descriptor is treated as a concept node, with relationships extracted from three relationship types: *USE* (preferred term substitution), *UF* (used-for: inverse of *USE*), and *RT* (related terms: captures substitutable words like "simple"/"approximate").

For each concept, its *Related Descriptors* are extracted and both key and related phrases are lemmatized to match the lemmatized corpus tokens. Co-occurrence counts between lemmatized word pairs are accumulated across all thesaurus entries, giving a raw frequency of how strongly two words are associated within the thesaurus concept space.

At query time, each query token is indexed in the thesaurus index. Since the thesaurus has limited coverage relative to the corpus vocabulary, query expansion is only applied when a match exists; OOV terms (both corpus words absent from the thesaurus and thesaurus words absent from the corpus vocabulary) are skipped. To suppress spurious low-frequency matches, only the top-2 related terms per query token that meet a global minimum count threshold k (a tunable hyperparameter) are appended to the query, controlling the precision-recall tradeoff of expansion. In the subsequent comparison study, we compare $k=5$ and $k=8$ with the baseline model (which used lemmatizer).

3.4 Comparison Study of candidate improvements

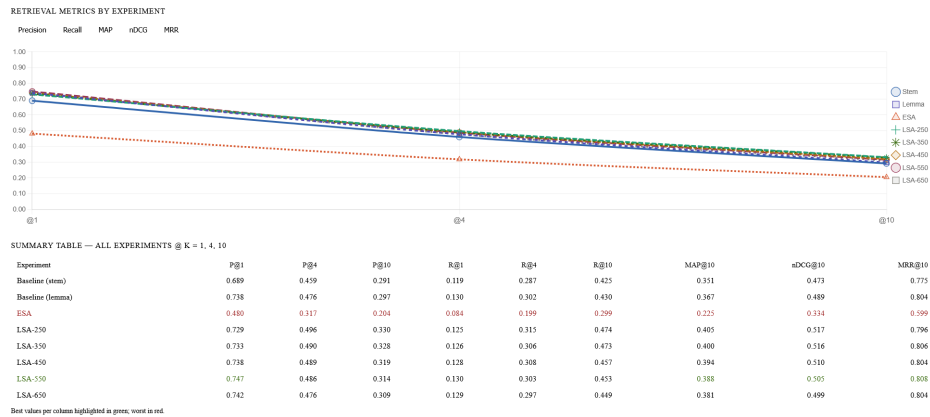


Fig. 5: Retrieval metrics by Experiment

HYPOTHESIS TESTS

H1 — Lemmatization outperforms stemming
 Baseline (lemma) vs Baseline (stem) — one-tailed Wilcoxon signed-rank test

Hypothesis rejected — no statistically significant difference

Metric	Statistic: W	p-value	Significant?
Precision @1	11357.5	0.1974	No
Recall @1	11357.5	0.1899	No
F-score @1	11357.5	0.1899	No
Precision @4	11321.5	0.1362	No
Recall @4	11217.0	0.1102	No
F-score @4	11258.5	0.1184	No
Precision @10	11316.5	0.1402	No
Recall @10	11369.0	0.1561	No
F-score @10	11320.5	0.1417	No

H2 — ESA underperforms the lemma baseline
 ESA vs Baseline (lemma) — one-tailed Wilcoxon signed-rank test

Hypothesis accepted — significant at all cutoffs

Metric	Statistic: W	p-value	Significant?
Precision @1	7215.0	1.00×10^{-8}	Yes
Recall @1	7210.0	5.66×10^{-8}	Yes
F-score @1	7210.0	5.66×10^{-8}	Yes
Precision @4	5478.0	5.78×10^{-16}	Yes
Recall @4	5330.5	2.91×10^{-16}	Yes
F-score @4	5219.0	1.20×10^{-16}	Yes
Precision @10	4791.0	3.94×10^{-16}	Yes
Recall @10	4779.0	3.92×10^{-16}	Yes
F-score @10	4689.0	1.83×10^{-16}	Yes

H3 — LSA (550 topics) outperforms the lemma baseline
 LSA-550 vs Baseline (lemma) — one-tailed Wilcoxon signed-rank test

Partially accepted — significant only at rank ≥ 8 (i.e. @10)

Metric	Statistic: W	p-value	Significant?
Precision @1	12494.0	0.8021	No
Recall @1	12504.0	0.8110	No
F-score @1	12504.0	0.8110	No
Precision @4	11823.5	0.3338	No
Recall @4	11937.5	0.4014	No
F-score @4	11940.0	0.4066	No
Precision @10	9498.5	0.0005	Yes
Recall @10	9434.5	0.0004	Yes
F-score @10	9442.5	0.0004	Yes

Fig. 6: Hypothesis tests for notable experiment pairs

- Despite lemmatization showing consistently higher numbers at all metrics and cutoff points, the differences are not statistically significant (all p-values ≥ 0.10). Both are viable preprocessing choices.
- ESA performs substantially worse than the lemma baseline at all cutoff points, and the effect is overwhelmingly significant (p-values in the range 10^{-8} to 10^{-16}). The performance gap widens at deeper cutoffs, suggesting that ESA struggles to rank relevant documents even moderately well.
- LSA-250 to LSA-650 all improve recall and $MAP@10$ over lemma baseline, with LSA-550 showing the best performance in most of the metrics chosen. However, this advantage only becomes statistically significant at deeper cutoff points ($k \geq 8$). At $k=1$ and $k=4$, LSA's gains could plausibly be noise ($p - values > 0.4$). The sweet spot for topic count appears to be 500–600, since performance degrades slightly as topics increase beyond that.

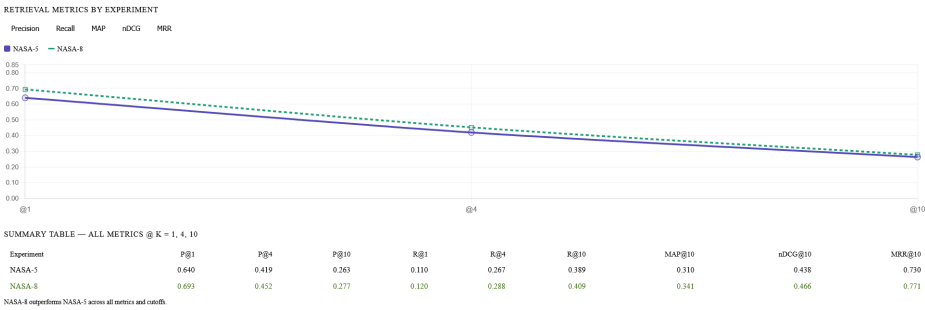


Fig. 7: Hypothesis tests for two select "NASA" pairs

HYPOTHESIS TESTS

H1 — NASA baseline underperforms the lemma baseline
Baseline (NASA-8) vs Baseline (lemma) — one-tailed Wilcoxon signed-rank test

Partially accepted — significant at @4 and @10, not @1

Metric	Statistic W	p-value	Significant?
Precision @1	11613.0	0.2063	No
Recall @1	11617.0	0.2072	No
F-score @1	11617.0	0.2072	No
Precision @4	10763.5	0.0319	Yes
Recall @4	10744.0	0.0303	Yes
F-score @4	10740.5	0.0300	Yes
Precision @10	9399.0	0.0004	Yes
Recall @10	9429.0	0.0004	Yes
F-score @10	9399.0	0.0004	Yes

H2 — NASA baseline outperforms ESA
Baseline (NASA-8) vs ESA — one-tailed Wilcoxon signed-rank test

Hypothesis accepted — significant at all cutoffs

Metric	Statistic W	p-value	Significant?
Precision @1	8236.5	1.82×10^{-4}	Yes
Recall @1	8134.5	1.21×10^{-4}	Yes
F-score @1	8134.5	1.21×10^{-4}	Yes
Precision @4	6561.5	1.67×10^{-9}	Yes
Recall @4	6292.0	3.55×10^{-11}	Yes
F-score @4	6270.0	3.05×10^{-11}	Yes
Precision @10	6089.5	9.73×10^{-11}	Yes
Recall @10	5906.5	4.35×10^{-11}	Yes
F-score @10	5824.0	1.53×10^{-11}	Yes

Fig. 8: Hypothesis tests for two experiment pairs (NASA-8 with the lemmatized baseline and NASA-8 with ESA

NASA-8 underperforms the lemmatized baseline in all metrics for ranks greater than $rank@4$, but outperforms ESA in all metrics for all ranks.

3.5 Pseudo-Relevance Feedback (PRF): Bridging the Vocabulary Gap

Concept and Implementation The primary bottleneck identified in Part 4 was the system’s inability to see beyond surface-level keyword matching. To address this, we implemented Pseudo-Relevance Feedback (PRF) using the Rocchio algorithm. The intuition is straightforward: if our initial TF-IDF pass is reasonably accurate, the top- N retrieved documents should contain latent “gold” terms that the user omitted. By harvesting these terms and re-injecting them into the query, we can perform a second, more informed retrieval.

For instance, a query regarding “transonic aileron buzz” might initially fail to match documents using the technical term “shock-induced separation.” PRF identifies this co-occurrence in the top results and automatically bridges the gap for the second pass.

We employed the positive-only Rocchio formula [3]:

$$\vec{q}' = \alpha \cdot \vec{q} + \beta \cdot \frac{1}{|D_r|} \sum_{d \in D_r} \vec{d}$$

Here, D_r represents the set of top- N documents from the first pass. To maintain query focus and prevent “topic drift” (where generic aerodynamic terms might overwhelm the specific query intent), we keep only the top- T terms by weight. After preliminary testing, we settled on parameters $\alpha = 1.0$ and $\beta = 0.75$, evaluating three expansion configurations (N, T) : (5, 15), (10, 20), and (15, 30).

Performance Analysis Our experiments showed that the leanest configuration, PRF(5,15), yielded the most robust results. Larger feedback sets tended to introduce noise, as lower-ranked (and potentially irrelevant) documents from the first pass began to dilute the query vector. Table 5 summarizes the gains.

Table 5: Comparative Metrics: Baseline TF-IDF vs. PRF(5,15)

	P@k		R@k		MAP@k		nDCG@k		MRR@k	
k	Base	PRF	Base	PRF	Base	PRF	Base	PRF	Base	PRF
1	.693	.662	.121	.114	.121	.114	.559	.524	.693	.662
4	.449	.471	.280	.293	.250	.260	.480	.482	.766	.746
7	.350	.366	.364	.380	.298	.314	.473	.477	.771	.750
10	.287	.304	.421	.436	.324	.341	.482	.488	.774	.752

The data reveals a clear trade-off. From $k = 4$ onwards, PRF consistently outperforms the baseline, providing a +6% boost in Precision and a +5.2% lift in MAP at $k = 10$. Interestingly, MRR and nDCG@1 show slight declines. This is a classic PRF side effect: while expansion significantly improves the “middle” of the pack by finding previously missed relevant documents, it occasionally

reshuffles the very top results. If a relevant document was already at rank 1, the new weighted terms might push it to rank 2 or 3, which MRR penalizes heavily. However, for most users, the trade-off of finding more relevant material deeper in the list is well worth a minor shift at the top.

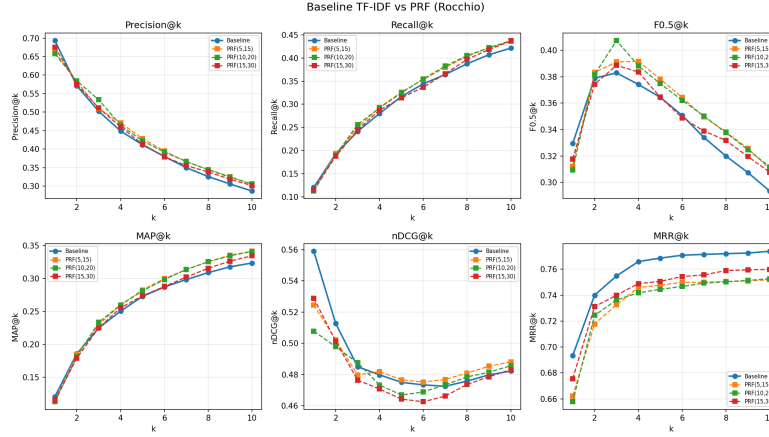


Fig. 9: Performance comparison of the Baseline vs. three Rocchio configurations.

Statistical Significance To ensure these gains weren't mere artifacts of the dataset, we performed a Wilcoxon signed-rank test [6] on per-query scores. The resulting p-values in Table 6 confirm the trend.

Table 6: Wilcoxon p-values (Baseline vs. PRF(5,15)). Significant values ($p < 0.05$) in **bold**.

k	Precision	Recall	$F_{0.5}$	AP	nDCG
1	0.194	0.138	0.138	0.138	0.087
4	0.004*	0.023*	0.022*	0.133	0.621
7	0.037*	0.034*	0.034*	0.029*	0.616
10	0.003*	0.021*	0.012*	0.006*	0.150

At lower ranks ($k \leq 3$), the variance is too high to claim significance. However, from $k = 4$ down to $k = 10$, the improvements in Precision, Recall, and Average Precision become statistically significant ($p < 0.05$). The nDCG metric remains the outlier; its focus on the first few positions prevents it from reaching significance, mirroring our observation about the slight rank-1 jitter caused by query expansion.

Ultimately, Rocchio-based PRF serves as a highly effective, "low-regret" upgrade to the VSM. It provides a significant boost in recall and precision exactly where the baseline was most prone to failure, all without requiring expensive model training or external semantic resources.

3.6 Spell Checker

The library SymSpell [5] was used to implement the spelling corrector in this project. It exploits the symmetric nature of the edit distance measure (for any pair of strings at edit distance k , deleting characters from either side generates overlapping candidate sets). Look-ups require only deletions (but not insertions or transpositions at query time), which is significantly faster than classic approaches.

In this implementation, the SymSpell instance considers and corrects words upto 2 character edits away from a known dictionary word. The dictionary loaded is *frequency_dictionary_en_82_765.txt*, a pre-built English frequency list containing 82,765 words with their corpus frequencies. Only the top candidate term is considered. The original word is unchanged if no close match is found.

3.7 BM25-Based Retrieval Improvement and Statistical Analysis

Motivation The baseline retrieval system uses the TF-IDF Vector Space Model (VSM). Although TF-IDF works well for keyword matching, it has a few important limitations. It increases term importance linearly with frequency, does not normalize document length effectively, and struggles with vocabulary mismatch in technical datasets such as Cranfield.

To improve retrieval quality, the BM25 ranking model was evaluated. BM25 improves over TF-IDF by introducing:

- Term frequency saturation, where repeated occurrences of a word contribute less additional importance after a point.
- Document length normalization, preventing very long documents from dominating retrieval scores.

The BM25 scoring function used is:

$$\text{score}(q, d) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{tf(t, d)(k_1 + 1)}{tf(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}$$

where:

- $tf(t, d)$ denotes the frequency of term t in document d
- $|d|$ denotes document length
- avgdl denotes average document length

The parameters used were:

$$k_1 = 1.5 \quad b = 0.75$$

Experimental Results BM25 was evaluated on the Cranfield dataset using the same preprocessing pipeline as the baseline system. Evaluation metrics were computed for values of k from 1 to 10.

Table 7: BM25 Retrieval Performance on Cranfield Dataset

k	Precision	Recall	$F_{0.5}$	MAP	nDCG	MRR
1	0.7333	0.1246	0.3434	0.7333	0.5989	0.7333
3	0.5422	0.2585	0.4123	0.5100	0.5070	0.7837
5	0.4302	0.3299	0.3809	0.4190	0.4835	0.7910
10	0.2884	0.4163	0.2944	0.3579	0.4846	0.7982

Observations BM25 showed strong early retrieval performance, achieving a Precision@1 score of approximately 0.733, meaning that nearly 73% of the queries retrieved a relevant document at the top rank.

As expected, recall increased with larger values of k , while precision gradually decreased because lower-ranked documents were also included in the retrieval results. The MRR score of approximately 0.798 at $k = 10$ indicates that relevant documents often appeared near the top of the ranked list.

Despite these improvements, several difficult queries still produced poor retrieval results. Examples included:

- “shock-sound wave interaction”
- “slender conic wing flow visualization”
- “laminar boundary layer compressible flow”

These failures suggest that BM25 still struggles with domain-specific terminology, phrase-level semantics, and vocabulary mismatch. Since BM25 mainly relies on lexical overlap, it cannot fully capture semantic relationships between technically related terms.

Wilcoxon Signed-Rank Test To determine whether the improvements over the baseline TF-IDF model were statistically significant, the Wilcoxon signed-rank test was performed for Precision@ k , Recall@ k , and $F_{0.5}@k$ for values of k from 1 to 10.

The null hypothesis used was:

H_0 : There is no significant difference between TF-IDF and BM25 retrieval performance.

The alternative hypothesis was:

H_1 : BM25 significantly improves retrieval performance over TF-IDF.

A significance threshold of:

$$\alpha = 0.05$$

was used.

Table 8: Wilcoxon Test Results: TF-IDF vs BM25

k	Precision p-value	Recall p-value	$F_{0.5}$ p-value
1	0.1871	0.1879	0.1879
2	0.0646	0.0695	0.0694
3	0.0410	0.0412	0.0395
5	0.0690	0.0629	0.0693
10	0.9041	0.8117	0.8133

The results show statistically significant improvements mainly at $k = 3$, where all p-values were below 0.05. This indicates that BM25 improves early ranking quality and retrieves more relevant documents among the top results.

However, for larger values of k , the improvements were not statistically significant. This suggests that although BM25 improves lexical ranking behavior, it does not completely solve the semantic retrieval challenges present in the Cranfield dataset.

These observations motivate the use of semantic retrieval approaches such as Latent Semantic Analysis (LSA) and domain-specific query expansion using the NASA thesaurus in the next stage of the project.

3.8 Unigram and Bigram weighting and Laplace Smoothing

The baseline retrieval system uses only the VSM-based TF-IDF score to compare and rank documents. As seen in class, if there was a way to detect commonly co-occurring phrases, then that would improve semantic retrieval. Also, since many words in our query may not have been seen in the document corpus, we will need to perform some kind of smoothing as seen in class.

This proposed method tries out the following:

- Laplace smoothing on individual word occurrences in the query
- Bigram modeling of the query followed by Laplace smoothing

The idea is to add these scores to the TF-IDF score to improve an existing ranking, and the complete scoring function is given as:

$$\text{score}(q, d) = \text{TF-IDF}_{\text{cosine}}(q, d) + \lambda_u \cdot B_u(q, d) + \lambda_b \cdot B_b(q, d)$$

where:

- $\text{TF-IDF}_{\text{cosine}}(q, d)$ denotes the standard TF-IDF cosine similarity score
- $B_u(q, d)$ denotes the normalized add-one-smoothed unigram language model bonus for document d given query q
- $B_b(q, d)$ denotes the normalized add-one-smoothed bigram language model bonus for document d given query q
- λ_u and λ_b are interpolation weights for unigram and bigram bonuses, respectively

We need to normalize the scores we get from the language modeling ideas so that they do not override the TF-IDF score itself.

Add-One Smoothing:

Add-one smoothing assigns a pseudo-count of 1 to every possible term, including those absent from the document. The smoothed conditional probability of observing term t in document d is:

$$P(t|d) = \frac{c(t, d) + 1}{|d| + |V|}$$

where:

- $c(t, d)$ denotes the count of term t in document d
- $|d|$ denotes the total count of all terms in document d (document length)
- $|V|$ denotes the total vocabulary size (number of unique terms)

Unigram Language Model Score:

The add-one-smoothed unigram language model score for a query q given document d is:

$$\text{LM}_{\text{unigram}}(q, d) = \sum_{t \in q} c(t, q) \cdot \log \left(\frac{c(t, d) + 1}{|d| + |V_{\text{unigram}}|} \right)$$

where $c(t, q)$ is the count of term t in the query, $|d|$ is the lexical term count in the document, and $|V_{\text{unigram}}|$ is the total unigram vocabulary size.

The score $B_u(q, d)$ has to be min-max normalized:

$$B_u(q, d) = \frac{\text{LM}_{\text{unigram}}(q, d) - \min_d \text{LM}_{\text{unigram}}(q, d)}{\max_d \text{LM}_{\text{unigram}}(q, d) - \min_d \text{LM}_{\text{unigram}}(q, d)}$$

Bigram Language Model Score:

For a sequence of tokens $[t_1, t_2, \dots, t_n]$, the bigrams are $[t_1-t_2, t_2-t_3, \dots, t_{n-1}-t_n]$.

The add-one-smoothed bigram language model score is:

$$\text{LM}_{\text{bigram}}(q, d) = \sum_{b \in \text{bigrams}(q)} c(b, q) \cdot \log \left(\frac{c(b, d) + 1}{|d|_{\text{bigram}} + |V_{\text{bigram}}|} \right)$$

where:

- b ranges over bigrams extracted from the query
- $c(b, q)$ is the count of bigram b in the query
- $c(b, d)$ is the count of bigram b in the document
- $|d|_{\text{bigram}}$ is the total bigram count in the document
- $|V_{\text{bigram}}|$ is the total bigram vocabulary size

The bigram score $B_b(q, d)$ is similarly normalized:

$$B_b(q, d) = \frac{\text{LM}_{\text{bigram}}(q, d) - \min_d \text{LM}_{\text{bigram}}(q, d)}{\max_d \text{LM}_{\text{bigram}}(q, d) - \min_d \text{LM}_{\text{bigram}}(q, d)}$$

Interpolation Weights:

The following standard parameter values were used throughout evaluation:

$$\lambda_u = 0.1 \quad \lambda_b = 0.1$$

The code allows for other weights, and is quite standalone from the other parts of the repository, but we did not do an extensive study on a hold-out dataset for the value of these weights.

We add these scores at the end of a ranking done by TF-IDF, and then re-rank. Evaluation metrics were computed for values of k ranging from 1 to 10, where k denotes the number of top-ranked documents considered.

Table 9 summarizes the observed retrieval performance of the proposed add-one-smoothed unigram-bigram hybrid model.

Table 9: TF-IDF + Add-One Unigram-Bigram Hybrid Retrieval Performance on Cranfield Dataset

k	Precision	Recall	$F_{0.5}$	MAP	nDCG	MRR
1	0.7422	0.1244	0.3440	0.7422	0.6000	0.7422
2	0.6244	0.2023	0.4055	0.6044	0.5359	0.7822
3	0.5393	0.2552	0.4086	0.5114	0.5033	0.7926
4	0.4722	0.2909	0.3923	0.4523	0.4884	0.7981
5	0.4267	0.3246	0.3768	0.4169	0.4814	0.7999
6	0.3859	0.3450	0.3557	0.3904	0.4761	0.8029
7	0.3530	0.3635	0.3364	0.3744	0.4736	0.8042
8	0.3306	0.3867	0.3237	0.3674	0.4767	0.8058
9	0.3101	0.4074	0.3107	0.3623	0.4803	0.8073
10	0.2956	0.4291	0.3017	0.3606	0.4865	0.8073

Several important retrieval improvements can be observed from the hybrid model evaluation results:

- Baseline TF-IDF Precision@1: 0.6844; Hybrid Precision@1: 0.7422
- Baseline TF-IDF Precision@3: 0.5111; Hybrid Precision@3: 0.5393
- Baseline TF-IDF MAP@10: 0.3467; Hybrid MAP@10: 0.3606
- Baseline TF-IDF nDCG@10: 0.4709; Hybrid nDCG@10: 0.4865
- Baseline TF-IDF MRR@10: 0.7687; Hybrid MRR@10: 0.8073

But there are still some examples of queries that continue to produce suboptimal results, such as:

- shock wave interaction with slender body
- laminar boundary layer flow separation
- compressible flow over conic geometry
- heat transfer in high-speed flow

These failures could have happened because of lack of semantics and because we do not have any idea of domain-specific synonymy, which is why we thought that ESA/LSA/NASA dataset retrieval is a good idea.

Statistical Significance Testing To assess whether improvements are statistically significant or potentially due to random variation, a Wilcoxon signed-rank test was conducted for each metric (Precision, Recall, $F_{0.5}$) using per-query metric values.

The null hypothesis used was:

H_0 : There is no statistically significant difference between the baseline TF-IDF system and the standalone TF-IDF + add-one-smoothed unigram-bigram hybrid retrieval system.

The alternative hypothesis was:

H1: The standalone TF-IDF + add-one-smoothed unigram-bigram hybrid system produces statistically significant retrieval improvements over the baseline TF-IDF system.

A significance threshold of $\alpha = 0.05$ was used. If the p-value obtained from the Wilcoxon test is less than 0.05, the null hypothesis is rejected in favor of the alternative hypothesis.

Table 10 summarizes the p-values obtained from the Wilcoxon signed-rank tests.

Table 10: Wilcoxon Signed-Rank Test Results: TF-IDF vs Standalone TF-IDF + Add-One Unigram-Bigram Hybrid

k	Precision p-value	Recall p-value	$F_{0.5}$ p-value
1	0.1132	0.1176	0.1176
2	0.0204	0.0270	0.0260
3	0.0463	0.0497	0.0479
4	0.0619	0.0696	0.0693
5	0.2489	0.2112	0.2200
6	0.8600	0.8928	0.9060
7	0.7899	0.6550	0.6826
8	0.9398	0.9341	0.9379
9	0.8013	0.7738	0.7946
10	0.4626	0.5208	0.4947

The Wilcoxon signed-rank test results reveal that there is statistically significant improvement at earlier values of $k = 2$ and $k = 3$, marginal significance at $k = 4$, and no significant improvement at later ranks. But, this is not necessarily a bad thing because people tend to focus on the first few replies to their queries.

3.9 Formal Conclusions (for all models in all of the comparison studies)

- We fail to reject the probability that the baseline (lemmatizer) model performs similarly to the baseline (stemmer) model with respect to the metrics *precision@1*, *precision@4*, *precision@10*, *recall@1*, *recall@4*, *recall@10*, *f1score@1*, *f1score@4* and *f1score@10* under the following assumptions (BoW, length normalization issue ignored, OOV terms discarded, concept orthogonality) on the cranfield dataset.
- The LSA-550 model is better than the baseline (lemmatizer) model with respect to *precision@10*, *recall@10* and *f1score@10* under the following assumptions (BoW for the baseline, similar word co-occurrence for LSA, length normalization issue ignored, OOV terms discarded) on the cranfield dataset.
- The baseline (lemmatizer) model is better than the ESA model with respect to *precision@1*, *precision@4*, *precision@10*, *recall@1*, *recall@4*, *recall@10*, *f1score@1*, *f1score@4* and *f1score@10* under the following assumptions (BoW, length normalization issue ignored, OOV terms discarded, concept orthogonality) on the cranfield dataset.
- The baseline (lemmatizer) model is better than the NASA-8 model with respect to *precision@4*, *precision@10*, *recall@4*, *recall@10*, *f1score@4* and *f1score@10* under the following assumptions (BoW, ignoring length normalization issue, OOV terms discarded, concept orthogonality) on the cranfield dataset.
- The NASA-8 model is better than the ESA model with respect to *precision@1*, *precision@4*, *precision@10*, *recall@1*, *recall@4*, *recall@10*, *f1score@1*, *f1score@4* and *f1score@10* under the following assumptions (BoW, length normalization issue ignored, OOV terms discarded, concept orthogonality) on the cranfield dataset.
- The BM25 model ($k_1=1.5$, $b=0.75$) is better than the baseline TF-IDF (stemmer) model with respect to Precision@3, Recall@3, and $F_{0.5}@3$ in the task of document retrieval on the Cranfield dataset under the following assumptions: bag-of-words representation, Porter stemming, stopword removal, cosine/BM25 scoring, and OOV terms discarded.
- We fail to reject the hypothesis that the baseline TF-IDF (stemmer) model performs similarly to BM25 with respect to Precision@ k , Recall@ k , and $F_{0.5}@k$ for $k \in \{1, 2, 4, 5, 10\}$ in the task of document retrieval on the Cranfield dataset under the same assumptions. This indicates that BM25's improvements over TF-IDF are localised to early-to-mid ranking positions and do not generalise across all cutoffs.
- The PRF(5,15) model ($\alpha=1.0$, $\beta=0.75$, top-5 feedback documents, top-15 expansion terms) is better than the baseline TF-IDF (stemmer) model with respect to Precision@ k , Recall@ k , and $F_{0.5}@k$ for $k \in \{4, 7, 10\}$ in the task of document retrieval on the Cranfield dataset under the following assumptions: bag-of-words representation, Porter stemming, stopword removal, positive-only Rocchio expansion, and OOV terms discarded.

- The PRF(5,15) model is better than the baseline TF-IDF (stemmer) model with respect to MAP@ k for $k \in \{7, 10\}$ in the task of document retrieval on the Cranfield dataset under the same assumptions.
- The Unigram-Bigram Hybrid model ($unigram_{weight} = 0.1, bigram_{weight} = 0.1$, add-one smoothing) outperforms the baseline TF-IDF model with respect to Precision@ k , Recall@ k , and F0.5@ k for all $k = 1, \dots, 10$, with statistically significant improvement (Wilcoxon signed-rank, $p=0.05$) at $k = 2, 3$, in the task of document retrieval on the Cranfield dataset under the following assumptions: bag-of-words representation, Porter stemming, NLTK stopwords removal, and unigram/bigram language model scores computed over lexical tokens only.
- We fail to reject the hypothesis that the baseline TF-IDF (stemmer) model performs similarly to PRF(5,15) with respect to Precision@ k , Recall@ k , $F_{0.5}@k$, and MAP@ k for $k \in \{1, 3\}$, and with respect to nDCG@ k for all $k \in \{1, 3, 4, 7, 10\}$ in the task of document retrieval on the Cranfield dataset under the same assumptions. The non-significance of nDCG is attributed to PRF's tendency to reshuffle the top 1–2 ranks, which the logarithmic discount of nDCG penalises heavily while barely rewarding the gains at deeper positions.

4 Conclusion and Acknowledgements

Conclusion

This project applied core NLP concepts by systematically exploring the design, implementation, and evaluation of an Information Retrieval (IR) system. We first established a robust text preprocessing pipeline and a foundational Vector Space Model (VSM) using TF-IDF term weighting. Our rigorous evaluation on the Cranfield dataset exposed the inherent limitations of exact lexical matching. We observed how the baseline model struggled with the out-of-vocabulary (OOV) problem, word sense ambiguity, and the inability to capture multi-word compositional semantics through extensive failure analysis.

We attempted to bridge this semantic gap by implementing the following advanced techniques:

- Explicit Semantic Analysis with Wikipedia2014 data
- Latent Semantic Indexing
- Query-Expansion with NASA Thesaurus
- Pseudo-Relevance Feedback
- BM25-based Retrieval
- Unigram-Bigram hybrid retrieval system

Our analysis of the performance of our hypotheses led to a lot of learnings for each of us, and also improved the performance of the IR system from a simple token-based VSM to a semantically-aware robust retrieval system.

Acknowledgements

We express our sincere gratitude to Prof. Sutanu Chakraborti for his guidance, engaging lectures, and continuous support throughout the course. We also extend our heartfelt thanks to the Teaching Assistants of the course for their prompt assistance and constructive feedback, helping us navigate the complexities of the programming assignments and this final evaluation.

Finally, we would like to acknowledge the collaborative effort and dedication of every member of our team, 5 Nearest Neighbours, without whom this project would not have been successful.

References

- [1] *Gensim: topic modelling for humans* — [radimrehurek.com](https://radimrehurek.com/gensim/). <https://radimrehurek.com/gensim/>.
- [2] *NASA Thesaurus* — [sti.nasa.gov](https://sti.nasa.gov/nasa-thesaurus/). <https://sti.nasa.gov/nasa-thesaurus/>.
- [3] J. J. Rocchio. *Relevance Feedback in Information Retrieval*. 1971.
- [4] *sentence-transformers/allenai-specter at main* — [huggingface.co](https://huggingface.co/sentence-transformers/allenai-specter/tree/main). <https://huggingface.co/sentence-transformers/allenai-specter/tree/main>.
- [5] *SymSpell/SymSpell/frequency_dictionary_en_82_765.txt at master · wolfgarbe/SymSpell* — [github.com](https://github.com/wolfgarbe/SymSpell/blob/master/SymSpell/frequency_dictionary_en_82_765.txt). https://github.com/wolfgarbe/SymSpell/blob/master/SymSpell/frequency_dictionary_en_82_765.txt.
- [6] *The Wilcoxon signed rank test for paired comparisons of clustered data - PubMed* — [pubmed.ncbi.nlm.nih.gov](https://pubmed.ncbi.nlm.nih.gov/16542245/). <https://pubmed.ncbi.nlm.nih.gov/16542245/>.